

Firewall Cheat Sheet (nftables & iptables)

See, add and remove the rules that decide which traffic a Linux server lets in. All commands need sudo.

Seeing the current rules

Command	Shows
<code>nft list ruleset · nft -a ...</code>	Every nftables rule · plus each rule's # handle (needed to delete it)
<code>iptables -L -n -v --line-numbers</code>	iptables rules with numbers and counters (ip6tables for IPv6)
<code>iptables -S · iptables-save</code>	Rules written as the commands that create them · every table, in save-file form
<code>iptables -v</code>	(nf_tables) = iptables is a front-end to nft; (legacy) = the old engine

Reading iptables -L -n -v --line-numbers

```
Chain INPUT (policy DROP 120 packets, 7200 bytes) ← policy: what happens to packets no rule matched
num  pkts bytes target    prot opt in  out source           destination
1     8312 612K ACCEPT all  --  lo  *  0.0.0.0/0         0.0.0.0/0
2     45M   61G ACCEPT all  --  *   *  0.0.0.0/0         0.0.0.0/0  ctstate RELATED,ESTABLISHED
3     214 12840 ACCEPT tcp  --  *   *  192.168.1.0/24    0.0.0.0/0  tcp dpt:22
4       37  2220 DROP  tcp  --  *   *  0.0.0.0/0         0.0.0.0/0  tcp dpt:3306
```

Column	Means	Column	Means
num	Rule number; first match wins	in / out	Interface; * = any
pkts bytes	Traffic that hit this rule	source	0.0.0.0/0 = anywhere
target	ACCEPT , DROP (silent), REJECT (says no)	dpt:22	Destination port 22
prot	tcp , udp , all (newer: 6 , 17 , 0)	ctstate	Connection state (ESTABLISHED = replies)

Reading nft list ruleset

```
table inet filter { ← inet = IPv4 and IPv6 together
  chain input {
    type filter hook input priority filter; policy drop; ← incoming traffic; default is drop
    ct state established,related accept ← replies to connections we started
    ip saddr 192.168.1.0/24 tcp dport 22 accept # handle 7 ← SSH from the LAN only
  }
}
```

Common tasks, side by side

Task	nftables	iptables
Add a rule (insert / -I puts it first): <code>nft add rule inet filter input rule · iptables -A INPUT rule</code>		
Allow replies	<code>ct state established,related accept</code>	<code>-m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT</code>
Allow SSH	<code>tcp dport 22 accept</code>	<code>-p tcp --dport 22 -j ACCEPT</code>
Manage rules (full commands)		
Drop the rest	<code>nft chain inet filter input '{ policy drop; }'</code>	<code>iptables -P INPUT DROP</code>
Delete a rule	<code>nft delete rule inet filter input handle 7</code>	<code>iptables -D INPUT 4 (by num)</code>
Save for reboot	<code>nft list ruleset > /etc/nftables.conf</code>	<code>netfilter-persistent save (Debian)</code>

Rules added by command are **gone after a reboot** until saved. For nft, also run `systemctl enable nftables`.

Friendlier front-ends (use one tool only; each overwrites hand-made rules)

ufw (Ubuntu)	Does	firewalld (RHEL, Fedora)	Does
<code>ufw status verbose</code>	Show rules	<code>firewall-cmd --list-all</code>	Show rules
<code>ufw allow 22/tcp</code>	Allow SSH	<code>firewall-cmd --permanent --add-service=ssh</code>	Allow SSH (saved)
<code>ufw enable</code>	Turn it on	<code>firewall-cmd --reload</code>	Apply saved rules

Remember: over SSH, allow SSH and replies **before** setting policy drop, and keep a console handy. A DROP policy survives `iptables -F`, so flushing the rules locks you out.